Ex. No.: 8
Date: 02/04/2025

PRODUCER CONSUMER USING SEMAPHORES

Aim: To write a program to implement solution to producer consumer problem using semaphores.

Algorithm:

1. Initialize semaphore empty, full and mutex.
2. Create two threads- producer thread and consumer thread.
3. Wait for target thread termination.
4. Call sem_wait on empty semaphore followed by mutex semaphore before entry into critical section.
5. Produce/Consume the item in critical section.
6. Call sem_post on mutex semaphore followed by full semaphore
7. before exiting critical section.
8. Allow the other thread to enter its critical section.
9. Terminate after looping ten times in producer and consumer Threads each.

Program Code:

```
# include <stdio.h>
# include <pthread.h>
# include <semaphore.h>
# include <unistd.h>
# define SIZE 10
int buffer[SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;

void producer() {
    int item;
    for (int i = 0; i < 10; i++) {
        item = rand() % 100;
        if ((in+1) % BUFFER_SIZE;
```

```
static int item=1;
sem_wait (&empty);
sem_wait (&mutex);
buffer[in]= item;
printf ("Producer produced : %d\n", item);
in = (in+1) % SIZE;
item++;
produced_count++;
sem_post (&mutex);
sem_post (&full); }
void consume(){
   if (consumed_count >=10){
         printf("consumption limit reached.\n");
         return; }
   sem_wait (&full);
   sem_wait (&mutex);
   int item = buffer[out];
   printf (" Consumer Consumed : %d\n", item);
   out = (out+1)% SIZE;
   Consumed_count++;
   sem_post (&mutex);
   sem_post (&empty); }
   int main(){
      int choice;
```

```
sem_init (&empty, 0, SIZE);
sem_init (&full, 0, 0);
sem_init (&mutex, 0, 1);
while(1) {
  if (produced_count >=10 && consumed_count >=10) {
  printf ("All items produced and consumed. Exiting.\n");
  break; }
printf ("\n1. PRODUCER\n 2. CONSUMER\n3. EXIT\n ENTER YOUR
    CHOICE:");
scanf ("%d", &choice);
switch (choice) {
   case 1:
     produce ();
     break;
   case 2:
     consume ();
     break;
   case 3:
     printf ("Exiting...\n")
     sem_destroy (&empty);
     sem_destroy (&full);
     sem_destroy (&mutex);
     exit (0);
   default:
     printf ("Invalid choice\n"); } }
sem_destroy (&empty);
sem_destroy (&full);
sem_destroy (&mutex);
return 0;
```

Sample Output:

1. Producer
2.Consumer
3.Exit
Enter your choice:1
Producer produces the item 1
Enter your choice:2
Consumer consumes item
1 Enter your choice:2
Buffer is empty!!
Enter your choice:1
Producer produces the item 1
Enter your choice:1
Producer produces the item 2
Enter your choice:1
Producer produces the item 3
Enter your choice:1
Buffer is full!!
Enter your choice:3

OUTPUT:

1. Producer
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE : 1

Producer Produced : 1

1. PRODUCER
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE : 1
Producer Produced : 2

1. PRODUCER
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE: 2
consumer Consumed : 1

1. PRODUCER
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE : 2
consumer consumed : 2

1. PRODUCER
2. CONSUMER
3. EXIT
ENTER YOUR CHOICE : 2
consumption limit reached.

Result:

Thus the code to implement solution to producer consumer problems using semaphores.